

Europe Code Week



Escape Room de algoritmos

Una idea *divertida, gamificada*, ideal para mostrar cómo la lógica y la programación sirven para resolver problemas reales.

IDEA GENERAL:

Concepto base:

Los estudiantes participarán en un *Escape Room* educativo donde deberán resolver una serie de **enigmas y retos lógicos** inspirados en **conceptos de programación y pensamiento computacional** (secuencias, condicionales, bucles, depuración...).

Cada acertijo superado desbloquea una “puerta” virtual hasta escapar del aula o completar la misión final.

Enfoque: aprendizaje activo, trabajo en equipo y gamificación para que comprendan la importancia de la lógica algorítmica en la vida real.

1. Duración y grupo destinatario

- **Duración total:** 2 sesiones de 1 hora y 30 minutos (puede adaptarse a una jornada de 2 horas).
- **Edad o nivel:** Estudiantes de **3º o 4º de ESO, FP Básica o 1º de DAW/DAM** (dependiendo del nivel de dificultad de los retos).

2. Objetivos de aprendizaje y competencias

Objetivos:

1. Comprender el valor del pensamiento algorítmico para resolver problemas.
2. Aplicar estructuras básicas de programación (secuencia, selección, repetición) de forma práctica.
3. Fomentar el trabajo colaborativo y la comunicación efectiva.
4. Desarrollar la creatividad y la capacidad de análisis.

5. Valorar la programación como herramienta para crear, no solo para usar tecnología.

Competencias trabajadas:

- Competencia digital.
- Competencia de aprender a aprender.
- Competencia en ciencia, tecnología, informática.
- Competencia social y cívica (trabajo en equipo y resolución pacífica de conflictos).

Habilidades adquiridas al finalizar:

- Diseñar y seguir algoritmos sencillos.
- Traducir instrucciones cotidianas a lenguaje lógico.
- Cooperar y coordinarse dentro de un grupo para lograr un objetivo común.

3. Recursos necesarios

- Aula con espacio libre para moverse o varias mesas “zona de reto”.
- Ordenadores o tablets (opcional, para pruebas digitales).
- Sobres o cajas con acertijos.
- Candados físicos o digitales (Google Forms con clave, por ejemplo).
- Material impreso: tarjetas con instrucciones, pistas, códigos QR, etc.
- Cronómetro o temporizador proyectado en la pizarra.
- Música ambiental tipo “Escape Room”.
- Proyector (para mostrar introducción, reglas y resultados).

4. Descripción detallada de la actividad

FASE 1: Introducción y contexto (15 minutos)

Objetivo: situar a los alumnos y despertar interés.

- Presentar brevemente qué es la *CodeWeek* y por qué aprender lógica y algoritmos puede ser divertido.
- Proyectar un breve vídeo motivador (puede ser de la web de CodeWeek o un tráiler de “Escape Room” educativo).
- Dividir la clase en **grupos de 4 o 5 alumnos**.
- Explicar las **reglas básicas del juego**:
 - Cada grupo debe resolver los retos por orden.
 - Solo pueden avanzar si completan correctamente cada uno.
 - Tienen un tiempo límite (20 minutos).

FASE 2: Desarrollo del Escape Room (60 minutos)

Cada equipo debe resolver 4 retos o “puertas” relacionadas con conceptos algorítmicos:

Puerta	Nombre del reto	Concepto trabajado	Descripción
1	El mensaje cifrado	Secuencia y orden	Deben ordenar una serie de instrucciones desordenadas para revelar un código secreto (ej. preparar un bocadillo o algoritmo de suma paso a paso).
2	El guardián lógico	Condicionales	Un enigma tipo “si... entonces...” donde deben elegir la ruta correcta (por ejemplo, solo abrir el sobre azul si el número es par).
3	El laberinto infinito	Bucles	Resolver un mini-laberinto repitiendo pasos hasta llegar a la salida, simulando un “while”.
4	Depura el sistema	Detección de errores	Encontrar errores en un algoritmo (en papel o pseudo-código) y corregirlos para obtener la clave final.

Variante digital:

Cada reto puede estar en un **Google Form**, **Genially**, o **Canva interactivo** con pistas ocultas o contraseñas.

FASE 3: Escape y reflexión final (15 minutos)

Objetivo: consolidar aprendizajes.

- Los equipos que logren escapar antes del tiempo reciben una “insignia digital CodeWeek”.
- Puesta en común: comentar qué lógica o estructura había detrás de cada reto.
- Debate breve: ¿en qué se parece esto a programar, les ha gustado y motivado interés en aprender más?

5. Evaluación

Criterios:

Aspecto	Descripción	Ponderación
Participación	Implicación y actitud durante el juego	30%
Resolución de problemas	Capacidad para aplicar lógica y cooperar	30%
Comprensión conceptual	Explicación de los algoritmos al final	20%
Creatividad y trabajo en equipo	Originalidad, roles dentro del grupo	20%

Instrumentos de evaluación:

- Rúbrica de observación del docente.
- Autoevaluación (breve formulario tipo “qué aprendí hoy”).
- Votación final por parte de los alumnos al equipo más colaborativo o ingenioso.

“Si enseñamos algoritmos jugando, estamos creando pensamiento lógico... sin que se den cuenta de que están aprendiendo.”

6. Ejemplos prácticos de las pruebas.

El mensaje cifrado:

Pasos desordenados:

- Remojas el cepillo con agua.
- Acomodas el cepillo sobre los dientes.
- Obtienes la pasta y destapas el tubo.
- Mueves el cepillo por todas las superficies durante 2 minutos.
- Ordenas el baño: tapas la pasta.
- Preparas el cepillo y el vaso.
- Giras el tubo y pones una pequeña cantidad en el cepillo.
- Dejas el cepillo enjuagado y escurrido.
- Revisas que todo quede limpio.
- Recoges el cepillo y lo llevas a la boca.
- Aclara con agua la boca.

Orden correcto:

1. **P**reparas el cepillo y el vaso.
2. **R**emojas el cepillo con agua.
3. **O**btienes la pasta y destapas el tubo.
4. **G**iras el tubo y pones una pequeña cantidad en el cepillo.
5. **R**ecoges el cepillo y lo llevas a la boca.
6. **A**comodas el cepillo sobre los dientes.
7. **M**ueves el cepillo por todas las superficies durante 2 minutos.
8. **A**clara con agua la boca.
9. **D**ejas el cepillo enjuagado y escurrido.

10. Ordenas el baño: tapas la pasta.

11. Revisas que todo quede limpio.

Iniciales: P-R-O-G-R-A-M-A-D-O-R → **PROGRAMADOR**

Depura el Sistema:

Planteamiento del reto:

Los alumnos son **analistas** de un sistema que está fallando.

Un algoritmo que debería **sumar los números pares del 1 al 10** está devolviendo el resultado equivocado.

Su misión: **detectar los errores, corregirlos y obtener el resultado correcto.**

Cuando lo consigan, el número obtenido será la **clave final**.

Materiales

- Hoja con el pseudocódigo erróneo (impreso o en pantalla).
- Acceso al “candado digital” (formulario) donde introducir la **clave correcta** (resultado del programa una vez depurado).



```
INICIO
suma = 0
PARA i DESDE 1 HASTA 10 HACER
  SI i ES IMPAR ENTONCES
    suma = suma + i
  FIN PARA
ESCRIBIR suma
FIN
```

Preguntas:

- ¿Qué hace realmente este código?
- ¿Qué condición debería usar para sumar solo los números **pares**?
- ¿Cuál será el resultado correcto una vez lo arreglés?

Solución:

Los números pares entre 1 y 10 son 2, 4, 6, 8 y 10 →

$$2 + 4 + 6 + 8 + 10 = \mathbf{30}$$

CLAVE: 30